COMP2022 Models of Computations: Turing Machines

Contents

1 Turing Machines **3**

1.1 Turing Machines in a nutshell . . . 3

1.2 Expression . . . 3

1.3 Design Tips . . . 5

1.3.1 Use blocks of code . . . 5

1.3.2 States store information . . . 5

1.3.3 View tape as multiple tracks . . . 5

1.4 Turing-Recognisable Languages . . . 6

1.5 Configurations . . . 7

1.5.1 TM computations . . . 7

2 Turing Machines are Robust **8**

2.1 Variation 1: Must-Move TMs . . . 8

2.2 Variation 2: Left-Bounded TM . . . 8

2.3 Variation 3: Multitape TMs . . . 9

2.4 Closure Properties . . . 10

3 Decidability **12**

3.1 Decidable Problems about Automata . . . 12

3.2 Decidable Problems about Regular Expressions, Automata, Context-Free Grammars 12

3.3 Acceptance Problem for TMs 12

3.4 Universal TM 13

4 Undecidability **14**

4.1 A Non-Recognisable Language 14

4.2 Non-Decidable Recognisable Language 14

4.3 Summary 15

5 Introduction to Computational Complexity P vs. NP **16**

5.1 Languages in P 16

5.2 Non-Deterministic TMs 17

5.2.1 Time complexity for NTMs 18

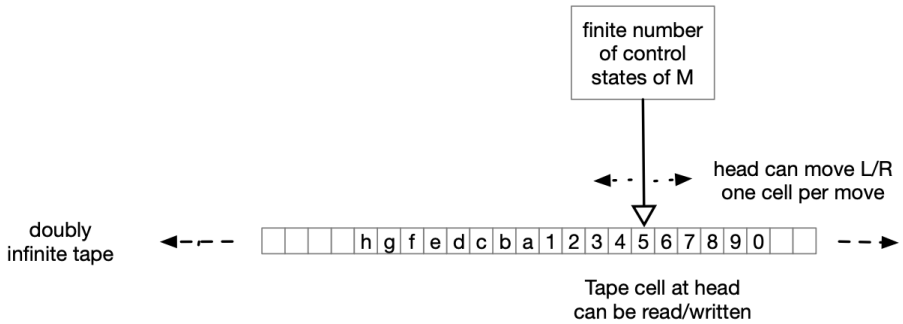
5.3 Languages in NP 18

5.3.1 Graphs (AK) 19

1 Turing Machines

1.1 Turing Machines in a nutshell

A Turing Machine (TM) M is like a program that can only use variables with finite domains, e.g., variable called *state* taking finitely many values. A TM has an infinite *tape* , made of *cells*, there is a single pointer (aka *head*) on the tape, the pointer can move left, right, or stay. The pointer can *read and write* symbols from a bigger *tape alphabet* Γ.



Initially the input string over *input alphabet* Σ is written on the tape. The machine can, depending on the state, decide to Halt (aka stop running) and "accept" or "reject", it is deterministic. A TM M *accepts* a string $u \in \Sigma^*$ if M starting with u on the input tape, with the pointer on the left-most symbol, from the initial state, runs and reaches an "accept" state.

The *language recognised by M (or language of M)* is the set of strings it accepts.

1.2 Expression

Each instruction is of the form

<current state> <current symbol> <new symbol> <direction> <new state>

The instruction

```
q1 a b r q2
```

tells the machine that if it is in state q_1 and sees the symbol a under the head, it should write the symbol b under the head, move one cell to the right, and change to state q_2. Symbols are from the tape alphabet Γ, which includes the input alphabet Σ and blank '_'.

Directions:

- Move one cell to the left L.
- Move one cell to the right R.

- Do not move $*$ (aka S).

The machine stops running when it reaches any state starting with *halt*, e.g. *halt*, *halt-accept*, *halt-reject*.

Example. TM recognising the language of a^*b^*.
Idea. Scan the tape, do not write anything, but just keep track that once we see a b, we do not again see an a.

```
q0 _ _ * halt-accept
q0 a a * q1
q0 b b * q2

q1 a a r q1
q1 b b r q2
q1 _ _ * halt-accept

q2 b b r q2
q2 _ _ * halt-accept
q2 a a * halt-reject
```

Example. TM recognising $\{0^n 1^n \mid n \geq 1\}$.

- Match leftmost 0 with leftmost 1, replace 0 by X and 1 by Y (so the TM does not lose its place), and repeat.
- If a 1 cannot be found in this way, reject (more 0s than 1s).
- If a 0 cannot be found in this way, go to the end and check there are no more characters.

```
; replace 0 by X and look for matching 1
; but if Y is seen, go for endgame
q0 0 X r q1
q0 Y Y r q3

; skip over 0's and Y's until 1 is found
; replace it by Y and start heading back to left
q1 0 0 r q1
q1 Y Y r q1
q1 1 Y l q2

; move left skipping 0's and Y's until the first X is found
; move right to look for leftmost 0
q2 Y Y l q2
q2 0 0 l q2
q2 X X r q0

; endgame
; make sure there are no extra 1's or 0's
q3 Y Y r q3
q3 _ _ * halt-accept
q3 0 0 * halt-reject
q3 1 1 * halt-reject
```

Definition (Formal Definition of Turing Machines). A *Turing machine* is a 7-tuple, $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where Q, Σ, Γ are all finite sets and

(a) Q is the set of states,

(b) Σ is the input alphabet not containing the *blank symbol* '_',

(c) Γ is the tape alphabet, where '_' $\in \Gamma$ and $\Sigma subsection \Gamma$,

(d) $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ is the transition function,

(e) $q_0 \in Q$ is the start state,

(f) $q_{\text{accept}} \in Q$ is the accept state, and

(g) $q_{\text{reject}} \in Q$ is the reject state, where $q_{\text{reject}} \neq q_{\text{accept}}$.

1.3 Design Tips

1.3.1 Use blocks of code

We already saw this in the previous example $\{0^n 1^n \mid n \geq 1\}$ and suppose that it is recognised by M_1. We now consider M_2 that recognises $\{0^n 1^n 0^n \mid n \geq 1\}$. We know that when M_1 reaches the accepting state, we have marked the string as n X's followed by n Y's, i.e. $X \ldots XY \ldots Y$. Hence, in M_2, we just need to simulate M_1 once again to match Y's and 0's, i.e. $X \ldots XY \ldots Y0 \ldots 0$.

1.3.2 States store information

A TM for $\{ww^R \mid w \in \{a, b\}^*\}$, where w^R is the reverse of w, could match the leftmost symbol with the rightmost symbol, erasing both. The state name can store the leftmost symbol while heading right to find the rightmost symbol, e.g. q_a means the leftmost symbol was an a, q_b means the leftmost symbol was a b.

1.3.3 View tape as multiple tracks

A TM for $\{ww \mid w \in \{a, b\}^+\}$ could first find and mark the midpoint of the input, and then use a matching process like before.

We view the tape as having two tracks. The first is for the input, the second is for marking the midpoint.

Formally, we change the tape alphabet. It now consist of a tape might look like

$$\dots \begin{bmatrix} _ \\ _ \end{bmatrix} \begin{bmatrix} _ \\ _ \end{bmatrix} \begin{bmatrix} a \\ _ \end{bmatrix} \begin{bmatrix} a \\ _ \end{bmatrix} \begin{bmatrix} b \\ _ \end{bmatrix} \begin{bmatrix} a \\ M \end{bmatrix} \begin{bmatrix} a \\ _ \end{bmatrix} \begin{bmatrix} b \\ _ \end{bmatrix} \begin{bmatrix} _ \\ _ \end{bmatrix} \begin{bmatrix} _ \\ _ \end{bmatrix} \dots$$

and the instructions will look like

```
q1 (a,M) (a,_) r q2
```

1.4 Turing-Recognisable Languages

Failure of Acceptance. A TM can fail to accept some input for one of two reasons.

(i) Its computation is *rejecting*, i.e., ends in a rejecting state.

(ii) Its computation is *diverging*, i.e., never reaches a rejecting or accepting state (and so infinite).

Definition (Turing-recognisable language).

(i) A language is *Turing-recognisable (aka recognisable)* if some TM recognises it.

(ii) A TM that halts on all inputs is called a *decider*.

(iii) A language is *Turing-decidable (aka decidable)* if some decider recognises it.

Various formalism of general computation have been proposed.

1. Turing machines (Alan Turing 1936)
2. Post systems (Emil Post)
3. μ-recursive functions (Kurt Gödel, Jacques Herbrand)
4. λ-calculus (Alonzo Church, Stephen C. Kleene)
5. Combinatory logic (Moses Schönfinkel and Haskell B. Curry)

Theorem. All these formalisms are equivalent, i.e., a problem is decidable in one formalism if and only if it is decidable by any other.

- This result strongly supports the idea that there is only one form of general computation.
- The *Church-Turing Thesis* says that the intuitive notion of algorithms equals Turing machine algorithms.

1.5 Configurations

Definition (Configurations). A *configuration* for a TM is a string $uqv \in \Gamma^* Q \Gamma^*$. Think of it as a "snapshot in time" of an execution of the TM. It represents the situation in which

1. q is the current state,
2. the tape content is uv (the infinite strings of blanks to the left and right of uv are not written),
3. the head is at the first symbol of v.

For example, the configuration $XXXq_1Y11$ represents the situation where the machine is in state q_1, the tape stores $XXXY11$, and the head is over the fourth cell that stores a Y.

There are some special configurations. For input string w, configurations of the form q_0w are called *start configurations*, e.g. q_0011.

Configurations of the form $uq_{\text{reject}}v$ are called *rejecting configurations*, e.g. $XYq_{\text{reject}}1$ (terminate before 1).

Configurations of the form $uq_{\text{accept}}v$ are called *accepting configurations*, e.g. $XXYYq_{\text{accept}}$.

1.5.1 TM computations

For configurations C, D write $C \vdash_M D$, read C *yields* D, if the TM can go from C to D in one step. E.g.

$$q_0011 \vdash Xq_111 \vdash q_2XY1 \vdash Xq_0Y1 \vdash XYq_31 \vdash XYq_{\text{reject}}1.$$

Write $C \vdash_M^* D$ if the TM M can go from C to D in any number of steps. E.g.

$$q_0011 \vdash_M^* XYq_{\text{reject}}1.$$

Theorem. The *language* $L(M)$ *recognised by the TM* M consists of all strings of the form w for which the computation starting with the start configuration q_0w ends in an accepting configuration.

$$L(M) = \{w \in \Sigma^* \mid \exists u, v \in \Sigma^* : q_0w \vdash_M^* uq_{\text{accept}}v\}$$

2 Turing Machines are Robust

Turing machines are fairly robust, i.e., variations and extensions of the basic model do not change the languages that can be recognised.

Let us define the *basic TMs* as TMs that are deterministic, has a single doubly-infinite tape, and can move left, right or stay.

Two machines are *equivalent* if they recognise the same language.

2.1 Variation 1: Must-Move TMs

Let us define TMs that disallow 'stay' as *must-move TMs*. Obviously, every must-move TM is a basic TM.

Theorem. Every basic TM is equivalent to a must-move TM.

Proof idea: simulate a basic TM by a must-move TM, in which replace each S-transition by an R-transition followed by an L-transition, i.e., replace

```
qi a b * qj
```

by

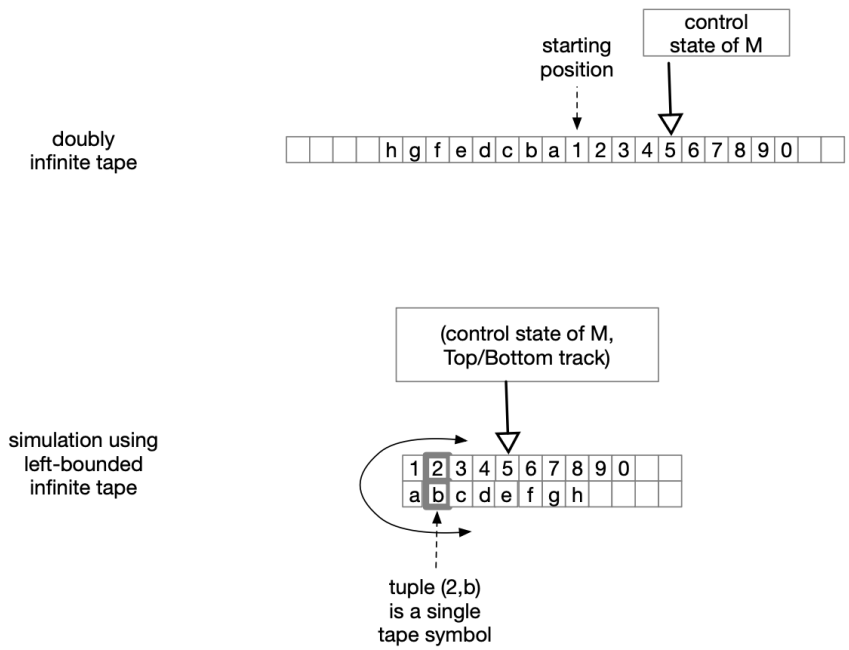
```
qi a b r qij
qij * * l qj
```

2.2 Variation 2: Left-Bounded TM

The head starts on the left-most square of the tape with head on the first letter of the input. When the transition function suggest to move left when the head is already at the left-most position, the head stays.

Theorem. Every basic TM is equivalent to a left-bounded TMs.

Proof idea: simulate a basic TM by a left-bounded one. In which split the left-bounded tape into two tracks, upper and lower. Upper track represents the right half of the tape from the initial head position. Lower track represents the left half of the tape. An extra state component keeps track of the half in which the head currently is. At the start, mark the leftmost position so the TM knows when to switch between upper and lower, q_{upper}, q_{lower}.



2.3 Variation 3: Multitape TMs

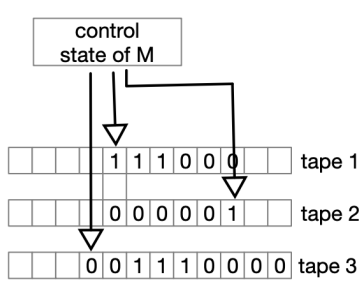
A multitape TM has multiple tapes, each with its own head for reading and writing. Initially, the input appears on tape 1, and the others start out blank. The heads move simultaneously (not necessarily same direction). The type of the transition function of a k-tape TM becomes

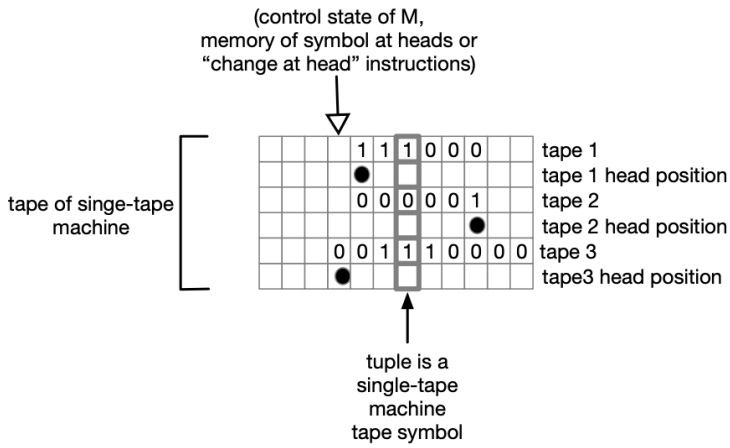
$$\delta : Q \times \Gamma^k \to \Gamma^k \times \{L, R, S\}^k \times Q.$$

Obviously every basic TM is a multitape TM with $k = 1$.

Theorem. Every multitape TM M has an equivalent basic TM B.

Proof idea: Split the tape into $2k$ many tracks of the single tape of B. For each tape of M, use one track to store tape contents, and one track to mark head position on that tape. Each transition of M is simulated by a series of transitions of B.





2.4 Closure Properties

We can think of a TM M as a procedure that takes an input x and outputs 1 (halt-accept), outputs 0 (halt-reject), or runs forever.

For instance:

```
def my_fun(x):
    return not M(x)
```

- outputs 1 if M halts and rejects x,
- outputs 0 if M halts and accepts x,
- runs forever if M runs forever on x.

Note that if M is a decider, the third option never happens.

Theorem (Complement). The complement of a decidable language is decidable.

Proof. Suppose that TM M decides language L. Build a TM from M as follows:

```
def my_fun(x):
    return not M(x)
```

This TM decides the complement of L.

Theorem (Union). The union of decidable languages is decidable.

Proof. Let L_1 and L_2 be languages that are decided by TMs M_1 and M_2 respectively. We construct a TM that decides $L_1 \cup L_2$ as follows:

```
def my_fun(x):
    if M1(x):
        return 1
    if M2(x):
        return 1
    return 0
```

If L_1 and L_2 are recognisable, the previous program

- does not recognise $L_1 \cup L_2$ (If M_1 runs forever, never have chance to run M_2);
- does not recognise ($x \notin L_1$, $x \in L_2$, $x \in L_1 \cup L_2$);
- recognises some set X where $L_1 \subseteq X \subseteq L_1 \cup L_2$.

Corollary (Intersection). If L_1 and L_2 are recognisable, then by De Morgan's Law for sets,

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

the intersection of decidable languages is decidable.

Theorem. A language is decidable if and only if it and its complement are recognisable.

Proof.

$\Longrightarrow$: follows from the closure properties of decidable languages.
$\Longleftarrow$: Say M_1 recognises L and M_2 recognises $\Sigma^* \setminus L$. We construct a decider for L as follows. On input w:

- Run M_1 and M_2 in parallel on w.
  - Use one tape for each machine.
  - At each step, simulate M_1 on tape 1 and M_2 on tape 2.
- If M_1 ever accepts, then accept; and if M_2 ever accepts then reject.

The point is that exactly one of M_1, M_2 must eventually accept w, and so we have built a decider for L.

3 Decidability

Recall: A TM is a *decider* if it halts on every input. A language L is called *Turing-decidable* if $L = L(M)$ for a decider M.

Encoding objects as strings

The input to a TM is always a string, but it can be useful to work with objects other than strings over Σ. We can encode almost any object as a string (integers, sets of integers, graphs, programs, TMs). The encoding of an object X may be written $\langle X \rangle$, $\mathrm{str}(X)$ or just X. The encoding of more than one object, say X and Y may be written $\langle X, Y \rangle$ or $X; Y$.

3.1 Decidable Problems about Automata

The *acceptance problem for DFAs* is the language

$$L_{\text{DFA-acceptance}} = \{D, w \mid D \text{ is a DFA that accepts } w\}.$$

This problem is decidable.

Here is a high-level description of a decider for this language:

1. Simulate D on word w.
2. If D ends in an accepting state then accept, else reject.

3.2 Decidable Problems about Regular Expressions, Automata, Context-Free Grammars

1. The acceptance-problems for DFA, NFA, RE and CFG are decidable.
2. The emptiness-problems for DFA, NFA, RE and CFG are decidable.
3. The equivalence-problems for DFA, NFA and RE are decidable.

3.3 Acceptance Problem for TMs

The *acceptance problem for TMs* is the language

$$L_{\text{TM-acceptance}} = \{M, w \mid M \text{ is a TM that accepts } w\}$$

- reads M as the string encoding of the source code of a TM M,
- this language is recognisable.

- To show this we will build a TM U that recognises it.

High level description of U: on input M, w simulate M on w accept if M enters q_{accept} and rejects if M enters q_{reject} (and diverge otherwise).

The TM U is called a *universal TM* since it can do what any other TM can do.

3.4 Universal TM

Implementation level description of U:

- Tape 1 holds the transition function δ_M of the input TM M.
- Tape 2 holds the simulated contents of M's tape.
- Tape 3 holds the current state of M, and the current position of M's tape head.
- U simulates M on input x one step at a time: in each step, it updates M's state and simulated tape contents and head position as dictated by δ_M. If every M halts and accepts or halts and rejects, then U does the same.

The universal TM U is a blueprint for an interpreter, it takes TMs as input and executes them.

This is similar to an interpreter of Python written in Python: a Python program that takes Python programs as input and executes them.

A universal TM can also be viewed as analogous to a computer.

The *acceptance problem for TMs* is the language

$$L_{\text{TM-acceptance}} = \{M, w \mid M \text{ is a TM that accepts } w\}.$$

TM U recognises $L_{\text{TM-acceptance}}$, but it does not decide it (since U is not a decider).

Theorem (Turing).

$$L_{\text{TM-acceptance}} = \{M, w \mid M \text{ is a TM that accepts } w\}$$

is not decidable.

4 Undecidability

4.1 A Non-Recognisable Language

Theorem.

$$L_{\text{Diag}} = \{\text{Source}_M \mid M \text{ is a TM that does not accept Source}_M\}$$

is not recognisable.

Proof. Let B be any TM. We will show $L(B) \neq L_{\text{Diag}}$ by showing that they disagree on the input $x = \text{Source}_B$.

There are two cases: either $\text{Source}_B \in L_{\text{Diag}}$ or not.

1. If $\text{Source}_B \in L_{\text{Diag}}$ then B does not accept Source_B, so $\text{Source}_B \notin L(B)$.
2. If $\text{Source}_B \notin L_{\text{Diag}}$ then B does accept Source_B, so $\text{Source}_B \in L(B)$.

Diagonalisation:

	Source_{M_1}	$\cdots$	Source_{M_k}
M_1	Reject		
$\vdots$		$\ddots$	
M_k			Accept

For all TMs, when they take their own source code as input, they do not accept. Therefore, these machines cannot possibly recognise L_{Diag}.

4.2 Non-Decidable Recognisable Language

Now we want to find a recognisable language that is not decidable. Recall that

$$L_{\text{TM-acceptance}} = \{\text{Source}_M, w \mid M \text{ is a TM that accepts } w\}$$

is recognisable. To prove that it is not decidable, we need to show that if it were possible to decide $L_{\text{TM-acceptance}}$ then it would be possible to recognise L_{Diag}, which we know is impossible as L_{Diag} is not recognisable as previously shown.

Theorem (Turing). There is no TM A that decides $L_{\text{TM-acceptance}}$.

Proof. We suppose there is a decider A for $L_{\text{TM-acceptance}}$ and we use it to build a TM B that recognises L_{Diag}.

```
def B(x):
    return not A(x,x)
```

For every TM M, the following are equivalent statements:

- B accepts Source_M,
- A does not accept $\text{Source}_M, \text{Source}_M$,
- M does not accept Source_M

In other words, B decides L_{Diag}, which we already saw is impossible. Hence, our assumption contradicts and therefore A cannot exist.

4.3 Summary

- $L_{\text{TM-acceptance}}$ is Turing-recognisable but not Turing-decidable.
- By the Church-Turing Thesis, this means that no program or computer can decide $L_{\text{TM-acceptance}}$.
- The same argument shows that $L_{\text{Python-acceptance}}$ is Python-recognisable but not Python-decidable.

There are two ways to show that a language L is undecidable:

1. Directly, using a diagonalisation argument.
2. Indirectly by assuming that there is a decider A for L, building an algorithm that using A decides a known undecidable problem.

5 Introduction to Computational Complexity P vs. NP

Recall that a *polynomial* (with integer coefficients) in the variable n is an expression of the form

$$\sum_{i=0}^{k} a_i n^i$$

where each a_i is an integer and $k \in \mathbb{Z}_0^+$. E.g., $2n^3 - 3n + 1$ is a polynomial.

Definition (Complexity). Time complexity of an algorithm or TM is denoted by a function

$$f(n) = \text{the largest number of steps taken by the algorithm/TM on any input of length } n.$$

For a TM, a single transition counts as a *step*.

If $f(n) = O(p(n))$ for some polynomial p, then we say that the algorithm/TM *runs in polynomial time*. For instance, M has time complexity $n \log n$ then M runs in polynomial time since $n \log n = O(n^2)$.

Example. $L = \{0^n 1^n \mid n \geq 1\}$.

On all strings of length n the TM makes at most $n/2$ passes, each pass scanning $n/2$ cells (the ones between the leftmost 0 and the leftmost 1). This shows that the time complexity is

$$\frac{n}{2} \times \frac{n}{2} = \frac{n^2}{4} = O(n^2).$$

On strings in the language it makes exactly $n/2$ passes; this shows that the time complexity is $\Theta(n^2)$.

5.1 Languages in P

Definition. Define **P** to be the collection of languages L that are decidable in polynomial time on deterministic TMs.

- Read "P" or "P Time" or "Poly Time" or "Polynomial Time" or "Deterministic Polynomial Time".
- **P** includes all the languages decided by linear-time algorithms, and quadratic-time algorithms, and cubic-time algorithms, etc.
- **P** is robust to certain changes in the model, notably multiple tapes.

P roughly corresponds to the problems that can be realistically solved on a computer.

Examples:

- Every regular language.
- Every context-free language.
- Some of the problems studied in this course, e.g., DFA membership, RE membership, DFA equivalence, CFG membership, CFG emptiness.

Note. The halting-problem is not in **P** because it is not decidable, but every language in **P** is decidable.

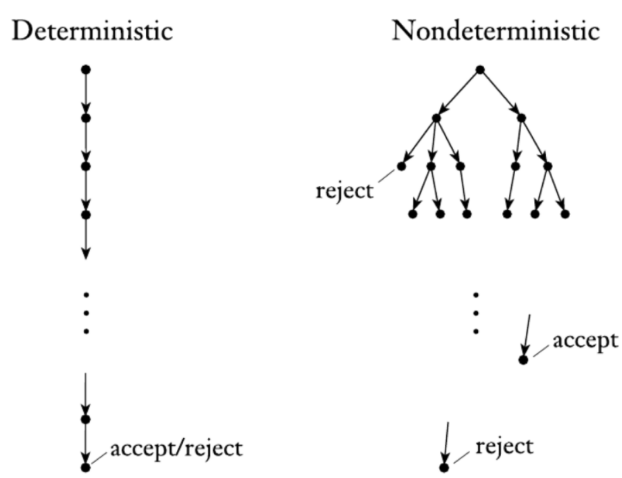
5.2 Non-Deterministic TMs

The type of transition function of a non-deterministic TM (NTM) N becomes

$$\delta : Q \times \Gamma \to P(\Gamma \times \{L, R, S\} \times Q).$$

For example, $(a', L, q') \in \delta(q, a)$ means if N is in state q and reads symbol a under the head, one of its possible transitions is to write a', change to state q', and move the head one cell to the left.

A computation of N on an input u is a tree, called the *computational tree*.



Definition. A NTM N accepts input $u \iff$ some branch of the computation tree of u has an accepting configuration.

So, if no branch of the tree has an accepting configuration (because each rejects or diverges) then N does not accept u.

Theorem. Every non-deterministic TM N has an equivalent deterministic TM D.

The idea is that D will search the computation tree of N. High-level description of TM D:

- D does a BFS of N's computation tree on given input.
- If it finds q_{accept} it accepts, otherwise it diverges.

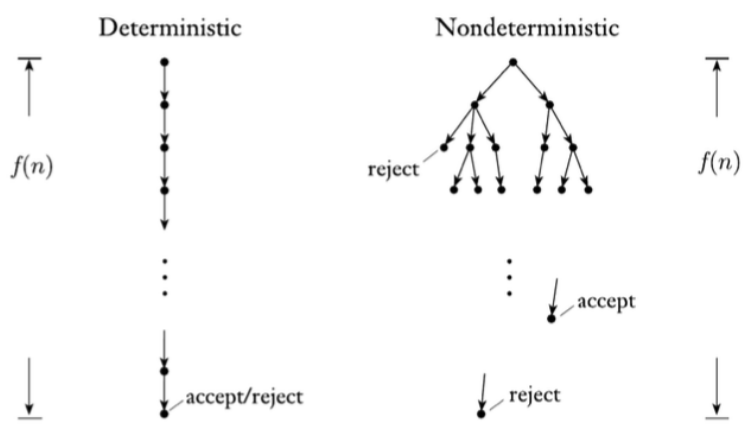
5.2.1 Time complexity for NTMs

Definition.

(a) An NTM N is a *decider* if on every input, every branch of its computation tree halts.

(b) The *time-complexity* of N is the function $f : \mathbb{N} \to \mathbb{N}$ where $f(n)$ is the maximum number of steps that N uses on any branch of its computation on any input of length n.

(c) If $f(n) = O(p(n))$ for some polynomial p then N *runs in polynomial time.*



5.3 Languages in NP

Definition. Define **NP** to be the collection of languages L that are decidable in polynomial time on *nondeterministic* Turing machines.

- Read "NP" or "Nondeterministic Polynomial time".

Fact: all languages in **P** are in **NP**.

5.3.1 Graphs (AK)

Recall that a graph G is a pair (V, E) where V is a set of *vertices* and $E \subseteq V \times V$ is a set of *edges*.

Example. A vertex represents a city and an edge represents a two-way road between two cities.

A non-empty graph is called a *clique* (aka "completely connected") if every pair of different nodes is connected by an edge.

Note. CLIQUE is in NP,

The CLIQUE problem:
Input: $G = (V, E)$ and $K \in \mathbb{Z}^+$.
Output: "Yes" if the graph contains K vertices that form a clique, and "No" otherwise.

To show that CLIQUE is in **NP**, we only need to give a polynomial time NTM that decides it.

Here is a high-level description of such a TM. On input $(V, E), K$:

1. Nondeterministically select a subset $C \subseteq V$ of K vertices.
2. Test whether every pair of different nodes in C is connected by an edge.
3. If yes, accept; else reject.